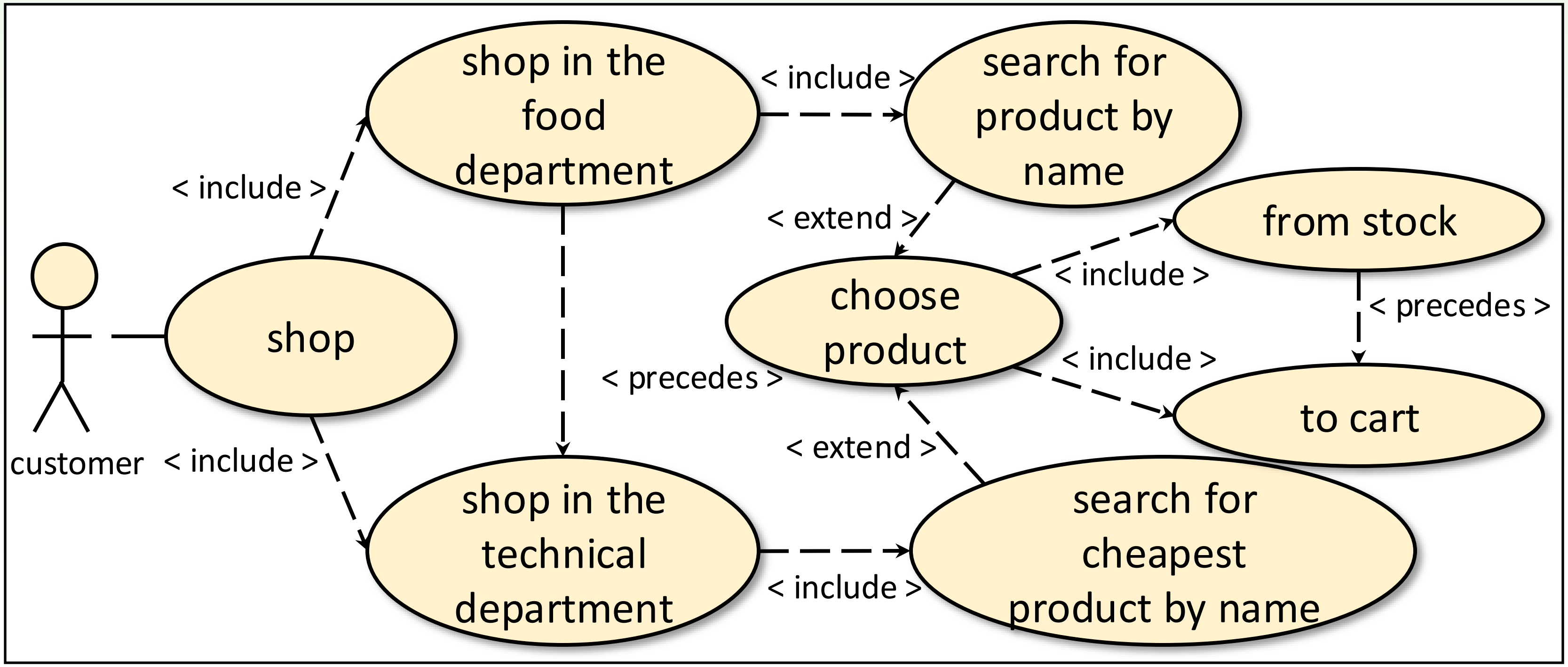
Shopping

A store consists of a grocery department and a technical department. Customers arrive with a shopping list containing the product names they intend to buy. In the store, they look for the products on their list in the following way. First, they look through the entire shopping list in the grocery department and take the products they find (put them in their basket). Then they repeat this in the technical department, but more thoughtfully: if an item on the shopping list appears multiple times in the same department, the cheapest one is chosen. We can assume that the same product is not present in both departments.

A store consists of a grocery department and a technical department. Customers arrive with a shopping list containing the product names they intend to buy. In the store, they look for the products on their list in the following way. First, they look through the entire shopping list in the grocery department and take the products they find (put them in their basket). Then they repeat this in the technical department, but more thoughtfully: if an item on the shopping list appears multiple times in the same department, the cheapest one is chosen. We can assume that the same product is not present in both departments.

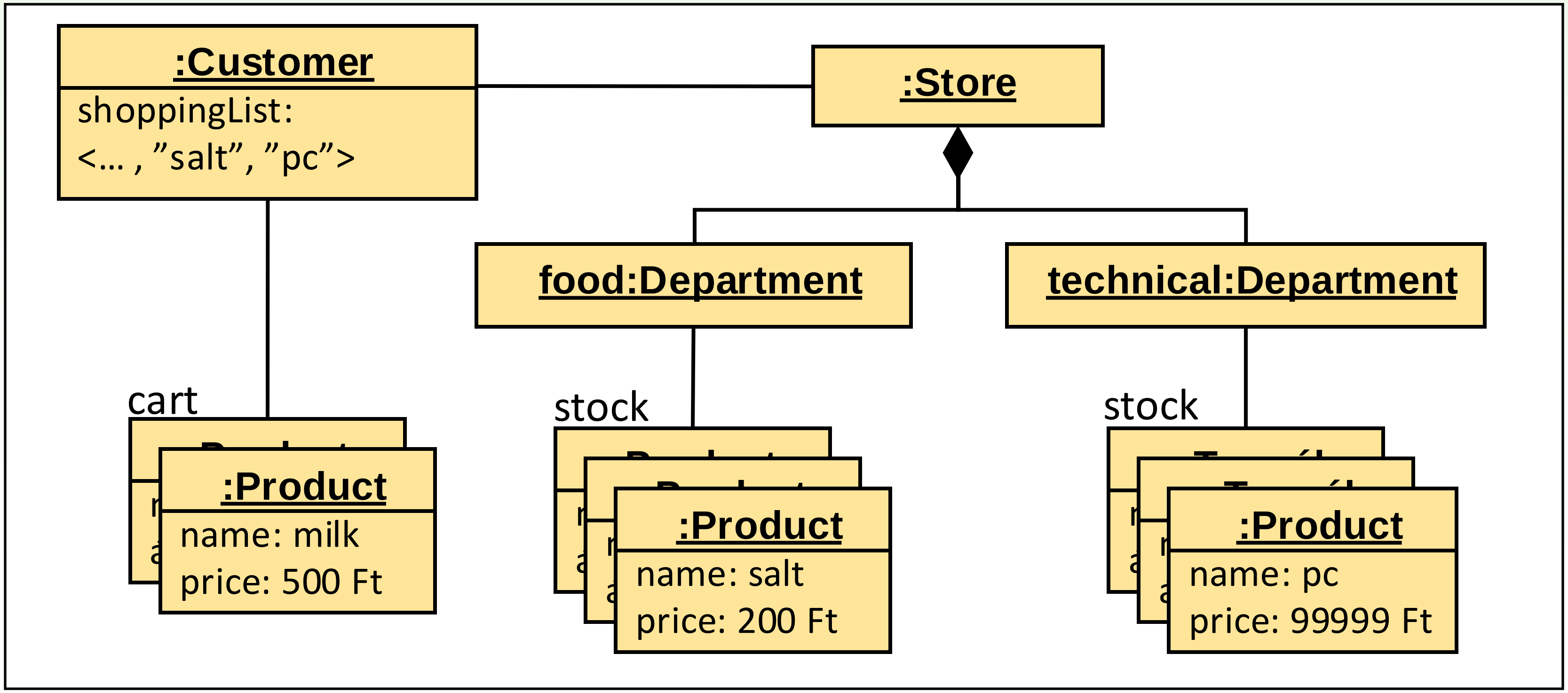
Shopping

A store consists of a grocery department and a technical department. Customers arrive with a shopping list containing the product names they intend to buy. In the store, they look for the products on their list in the following way. First, they look through the entire shopping list in the grocery department and take the products they find (put them in their basket). Then they repeat this in the technical department, but more thoughtfully: if an item on the shopping list appears multiple times in the same department, the cheapest one is chosen. We can assume that the same product is not present in both departments.



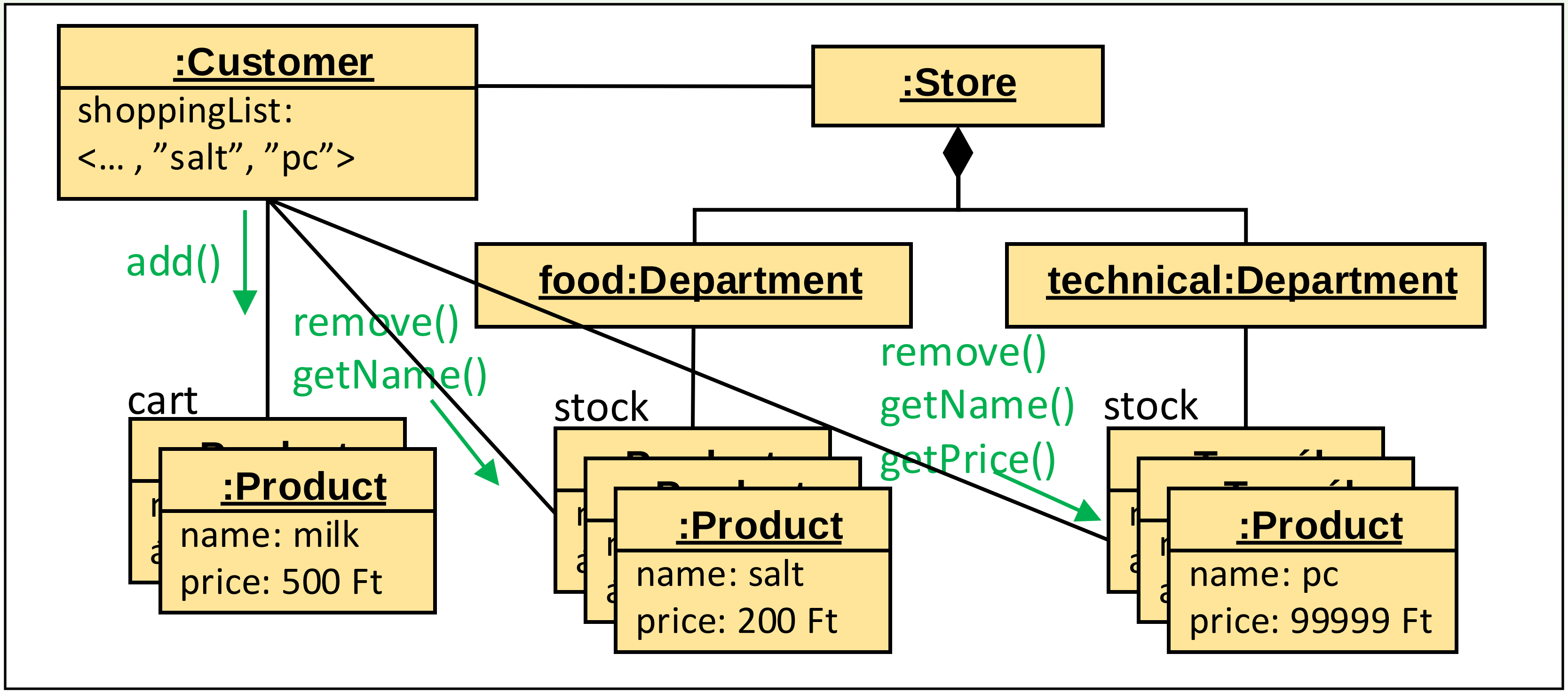
Shopping

A store consists of a grocery department and a technical department. Customers arrive with a shopping list containing the product names they intend to buy. In the store, they look for the products on their list in the following way. First, they look through the entire shopping list in the grocery department and take the products they find (put them in their basket). Then they repeat this in the technical department, but more thoughtfully: if an item on the shopping list appears multiple times in the same department, the cheapest one is chosen. We can assume that the same product is not present in both departments.

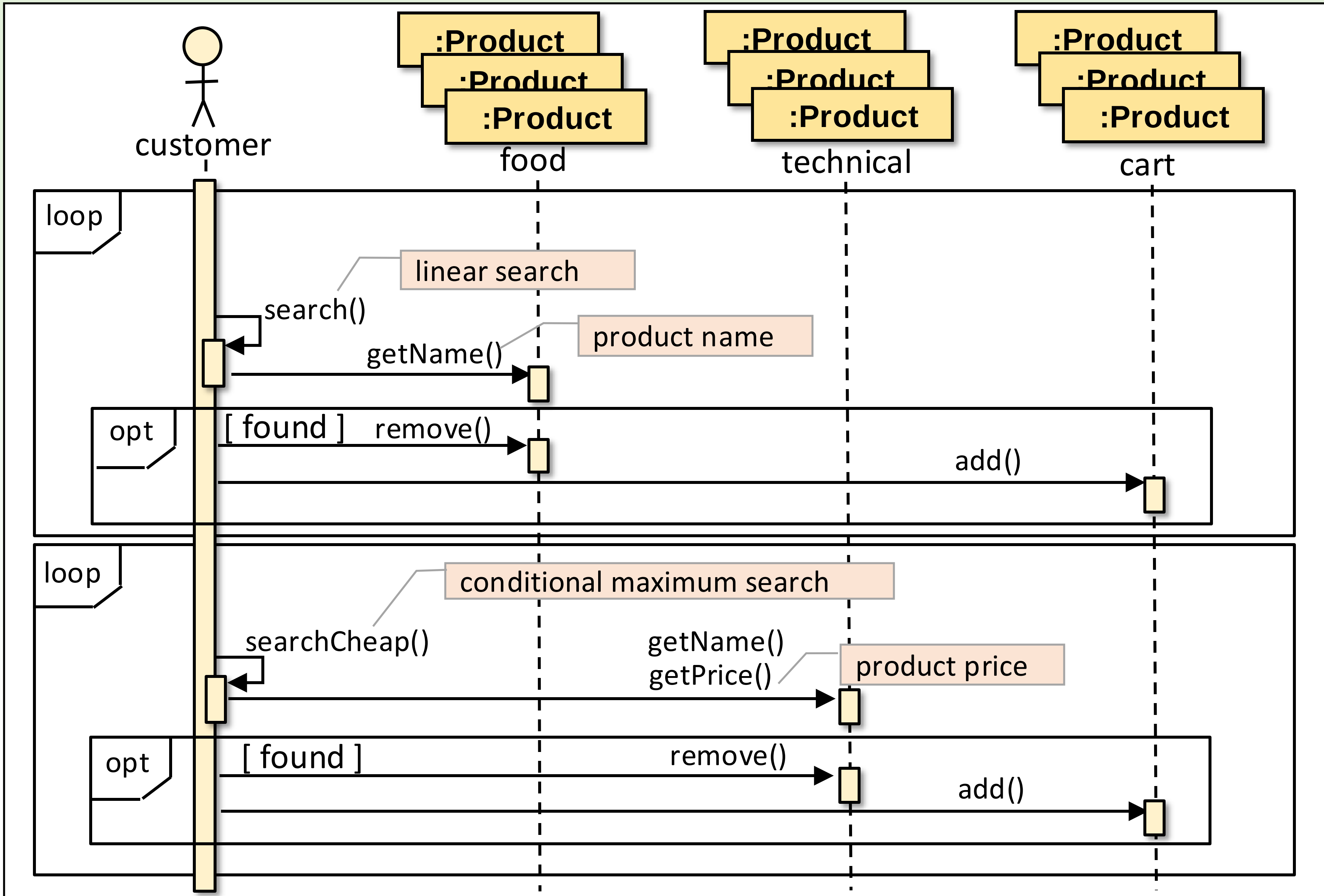


Shopping

A store consists of a grocery department and a technical department. Customers arrive with a shopping list containing the product names they intend to buy. In the store, they look for the products on their list in the following way. First, they look through the entire shopping list in the grocery department and take the products they find (put them in their basket). Then they repeat this in the technical department, but more thoughtfully: if an item on the shopping list appears multiple times in the same department, the cheapest one is chosen. We can assume that the same product is not present in both departments.

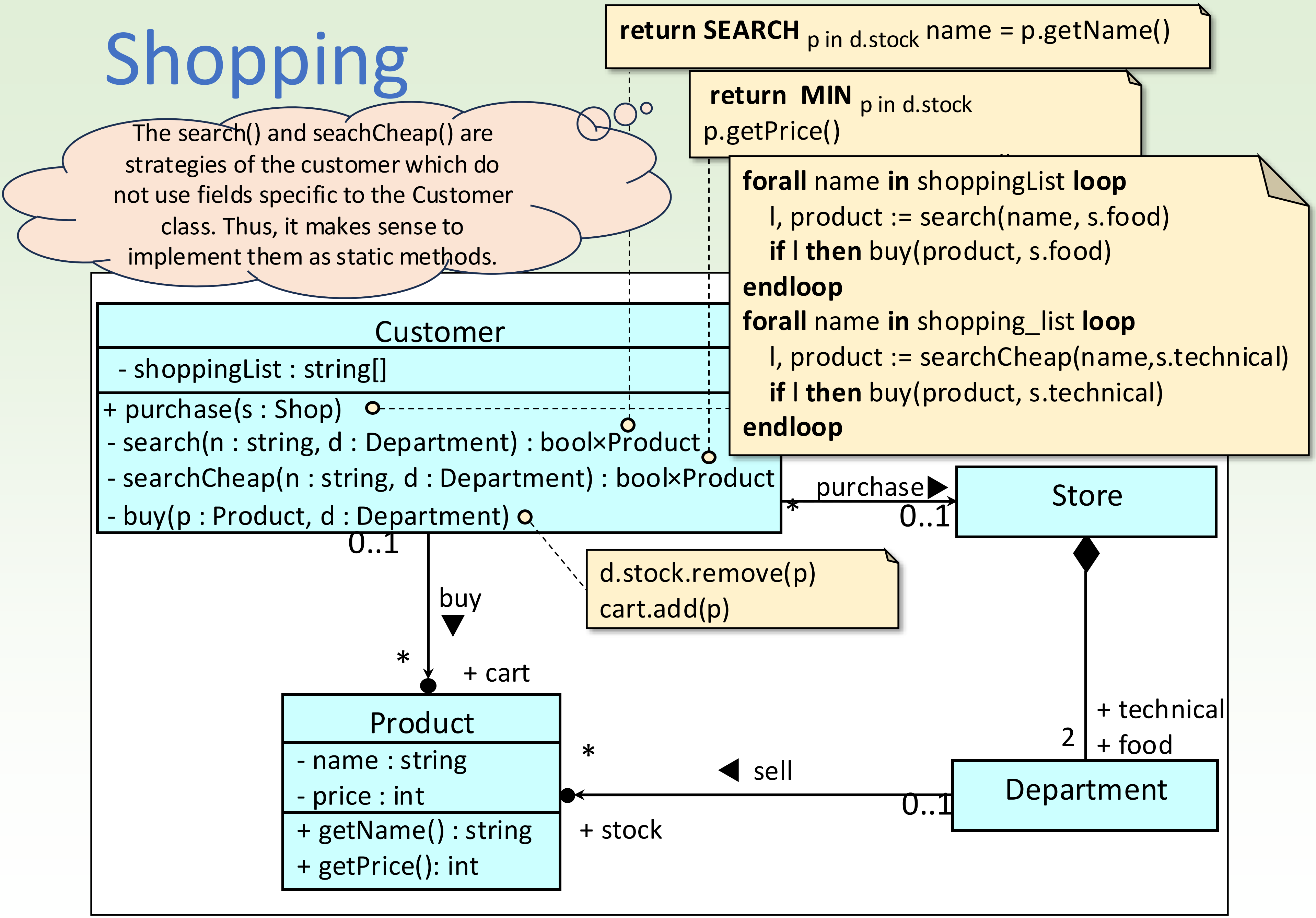


Shopping



:Product
:Product
:Product
customer
food
technical
cart
loop
linear search
search()
product name
getName()
opt
[found]
remove()
add()
loop
conditional maximum search
searchCheap()
getName()
getPrice()
product price
opt
[found]
remove()
add()

Shopping



return SEARCH p in d.stock name = p.getName()
return MIN p in d.stock p.getPrice()
The search() and seachCheap() are strategies of the customer which do not use fields specific to the Customer class. Thus, it makes sense to implement them as static methods.
forall name in shoppingList loop
l, product := search(name, s.food)
if l then buy(product, s.food)
endloop
forall name in shopping_list loop
l, product := searchCheap(name,s.technical)
if l then buy(product, s.technical)
endloop
Customer
- shoppingList : string[]
+ purchase(s : Shop)
- search(n : string, d : Department) : bool×Product
- searchCheap(n : string, d : Department) : bool×Product
- buy(p : Product, d : Department)
purchase
*
0..1
Store
0..1
buy
*
+ cart
d.stock.remove(p)
cart.add(p)
Product
- name : string
- price : int
+ getName() : string
+ getPrice(): int
*
sell
+ stock
0..1
Department
2
+ technical
+ food

ATM

At an ATM, customers line up to withdraw money. Customers have bank cards. A bank card is associated with a bank account and has a PIN code. A customer inserts their bank card, then enters the PIN code to verify authenticity. The customer then enters the amount they want to withdraw. If the amount is still positive after subtracting it from the customer's account balance, the ATM dispenses the amount. As a part of the process, the ATM requests the customer's account balance from a central office based on info found on the customer's card, and sends a transaction report to the customer's bank, which then deducts the amount from the customer's account.

Package delivery

A package delivery driver delivers packages from one location to PickPack points, installed at various fuel stations. This means, that they have the option to refuel at each delivery address. The packages are marked with their delivery address, which can be used to calculate the distance from the current location to the selected address (in kms).

The driver loads as many packages into the cargo area fully with packages, then proceeds as follows: selects an address for one of his packages (if the cargo area is empty, then the address is the driver's base location); checks the fuel level to see if it is sufficient for delivery (if not, he refuels); drives to the selected address (as a result, the fuel level decreases); then unloads the packages sent to that address.

The vehicle has a cargo and a fuel tank. The cargo can hold up to a specified number of packages. Refueling is possible up to a specified maximum fuel level. The vehicle's fuel consumption (in liters/km) is known.